나의 입출금 시스템 만들기

Code Files : 나의입출금시스템.ipynb

1. 개요와 학습 목표

1.1 만드는 내용

2026 년 1~3 월의 수입·지출 내역을 표로 만들고, 전체 수입과 지출, 항목별 지출 순위, 월별 남은 금액 분석. 분석 결과는 막대그래프와 CSV 파일로 저장.

Code 항목	완성 결과	확인 내용
1. 데이터/2.수입과 지출 구분	전체 수입·지출·남은 금액	총 수입·지출·남은 금액 확인
3. 표시열 만들기	표시열 추가	10 만원 이상 큰지출 작으면 일반지출
4. 항목별 지출 순위	항목별 지출 순위	지출이 가장 큰 항목 확인
5. 날짜에서 월을 꺼내고 피벗 만들기	월별 요약표	지출과 남은 금액이 큰 월 확인
6. 지출 합계 그래프 그리기	항목별 지출 합계 막대그래프	지출 규모 비교
	CSV 파일 2 개	항목별 지출,csv, 월별요약.csv

1.2 학습 목표 및 수행 내용

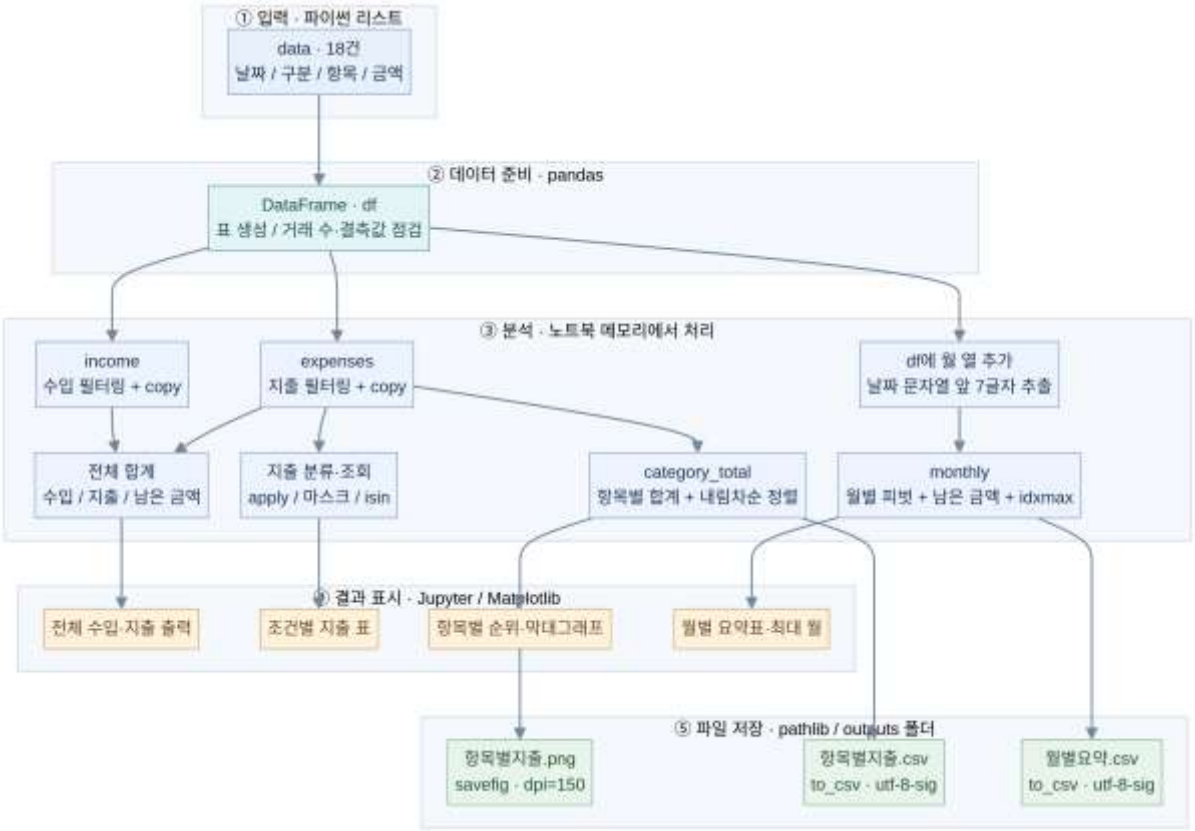
- 1. 데이터/2.수입과 지출 구분
  - 리스트를 DataFrame 으로 만들고 데이터의 기본 상태를 점검
  - 조건에 맞는 행을 선택하고 금액을 합산
- 3. 표시열 만들기

    - 함수와 apply()로 새로운 열을 생성.
- 4. 항목별 지출 순위
    - groupby()와 정렬로 항목별 합계와 순위를 구했음.
- 5. 날짜에서 월을 꺼내고 피벗 만들기
    - pivot_table()로 월별 수입·지출 요약표를 작성했음.
- 6. 지출 합계 그래프 그리기
    - 그래프와 CSV 로 결과를 저장했음.
- 미니 실습 : concat()과 merge()의 차이를 확인

1.3 전체 분석 흐름

데이터 준비 → 표 만들기·점검 → 수입/지출 분리 → 합계 계산 → 조건별 분류(표시열 만들기) → 항목별 집계 → 월별 집계 → 시각화·저장

1.4 시스템 아키텍처



구성 계층	주요 구성 요소	수행 역할	처리 결과
입력	파이썬 리스트 data	날짜·구분·항목·금액으로 거래 18 건을 입력	원본 거래 데이터
데이터 준비	pandas, df	DataFrame 을 생성하고 거래 건수·결측값을 점검	분석 기준 표
분석	income, expenses, category_total, monthly	수입·지출 필터링, 지출 분류, 항목별·월별 집계를 수행	전체 합계, 조건별 내역, 항목별 순위, 월별 요약
결과 표시	print(), display(), Matplotlib	숫자·표·막대그래프로 분석 결과를 표시	분석 결과
파일 저장	Path, to_csv(), savefig()	outputs 폴더에 이미지 1 개와 CSV 2 개를 저장	항목별지출.png, 항목별지출.csv, 월별요약.csv

핵심 데이터 경로

1. **전체 합계 경로:** data → df → income / expenses → sum() → 전체 수입·지출·남은 금액으로 처리.
2. **지출 상세·항목별 경로:** df → expenses 에서 분기. apply()·마스크·isin()으로 분류·조회 결과를 만듬, 별도로 groupby()·정렬을 통해 category_total 을 생성. category_total 에서 그래프와 항목별 CSV 를 만듬.
3. **월별 경로:** df → 월 열 추가 → pivot_table() → monthly → 남은금액·idxmax()로 처리. 전체 거래를 포함한 df 에서 수입과 지출 월별 집계. 월별 결과는 화면과 월별 CSV 에 출력.

미니 실습의 연결 위치

- **월별 거래 통합:** df → january / february → concat() → two_months 로 이어 붙임.
- **목표 비교:** monthly.reset_index() + goal → merge(on='월') → comparison 으로 연결. 월별 남은 금액과 목표금액의 차이를 계산
- **데이터 관계 주의:** income 과 expenses 는 df 를 필터링한 후 .copy()로 만든 별도 표. 이후 expenses 에 만든 표시 열이 df 에 자동 반영되는 구조가 아님, df 에

나중에 만든 월 열도 기존 expenses 에 자동 반영되지 않음. 아키텍처의 화살표는 실제 처리 단계 간 데이터 흐름을 나타냈으며, 실행이 끝난 뒤 자동 동기화되는 연결을 의미하지 않음.

요약: 거래를 하나의 데이터프레임으로 준비한 뒤 전체 합계·지출 분석·월별 분석으로 나누어 처리. 각 분석 결과를 화면에 표시하고, 항목별 합계와 월별 요약을 파일로 저장하는 구조

---

2. 데이터 준비와 기본 확인

2.1 환경 설정

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
from pathlib import Path
from IPython.display import display

OUTPUT = Path('outputs')
```

```
OUTPUT.mkdir(exist_ok=True)
```

도구	역할
pandas	표 형태의 데이터 처리와 분석
matplotlib.pyplot	그래프 생성과 저장
Path	파일·폴더 경로 처리
display()	노트북에서 표를 보기 좋게 출력

OUTPUT.mkdir(exist_ok=True)는 결과를 저장할 폴더를 만듦. 이미 폴더가 있어도 오류를 내지 않음. outputs 는 코드가 실행되는 현재 작업 폴더를 기준으로 한 상대 경로임.

2.2 데이터의 구조

한 행은 거래 한 건이며, 네 개의 열을 사용함.

열 이름	의미	예시
날짜	거래가 발생한 날짜	2026-01-01
구분	수입 또는 지출	수입, 지출
항목	거래의 종류	월급, 월세, 식비, 교통, 쇼핑, 통신
금액	거래 금액, 단위는 원	2000000

예제의 입력 규칙: 금액은 항상 양수로 기록하고, 구분 열로 수입과 지출을 구별함. 날짜는 YYYY-MM-DD 형식의 문자열임.

```
data = [
    ['2026-01-01', '수입', '월급', 2000000],
    ['2026-01-05', '지출', '월세', 600000],
    ['2026-01-10', '지출', '식비', 280000],
    ['2026-01-15', '지출', '교통', 70000],
    ['2026-01-20', '지출', '쇼핑', 150000],
    ['2026-01-25', '지출', '통신', 100000],
    ['2026-02-01', '수입', '월급', 2000000],
    ['2026-02-05', '지출', '월세', 600000],
    ['2026-02-10', '지출', '식비', 320000],
    ['2026-02-15', '지출', '교통', 80000],
    ['2026-02-20', '지출', '쇼핑', 300000],
    ['2026-02-25', '지출', '통신', 120000],
    ['2026-03-01', '수입', '월급', 2000000],
    ['2026-03-05', '지출', '월세', 600000],
    ['2026-03-10', '지출', '식비', 250000],
    ['2026-03-15', '지출', '교통', 60000],
```

```
    ['2026-03-20', '지출', '쇼핑', 100000],
    ['2026-03-25', '지출', '통신', 80000],
]

df = pd.DataFrame(data, columns=['날짜', '구분', '항목', '금액'])
```

```
display(df)
```

DataFrame 은 행과 열로 구성된 표임. df['금액']처럼 열 하나를 선택하면 Series 가 되고, df[['항목', '금액']]처럼 여러 열을 선택하면 DataFrame 이 됨.

2.3 분석 전 점검

```
print('내역 수:', len(df))
```

```
display(df.head())
print(df['구분'].value_counts())
```

```
print('빠진 값:', df.isna().sum().sum())
```

확인 항목	결과
전체 거래 수	18 건
수입 거래 수	3 건
지출 거래 수	15 건
결측값 수	0 개

head()는 기본적으로 처음 5 행을 보여 줌. value_counts()는 값별 등장 횟수를 셈. isna()는 결측 여부를 표시하고, 첫 번째 sum()은 열별 결측 개수, 두 번째 sum()은 전체 결측 개수를 계산함.

확인 항목: 1 월 월급과 월세의 행 번호 확인. 기본 인덱스 기준으로 각각 0, 1 임. 행 번호가 0 부터 시작한다는 점을 설명함.

3. 조건에 맞는 데이터 선택과 합계 계산

3.1 수입과 지출 분리: 불리언 마스크

```
income = df[df['구분'] == '수입'].copy()
```

```
expenses = df[df['구분'] == '지출'].copy()
```

이 코드는 다음 세 단계로 읽음.

4 df['구분']으로 구분 열을 선택함.

5 == '수입'으로 각 행이 조건에 맞는지 비교해 True 또는 False 를 만듦.

6 df[조건]으로 True 인 행만 선택함.

이처럼 행을 선택하는 참·거짓 조건을 **불리언 마스크**라고 부름. .copy()는 선택한 결과를 별도 데이터로 다룬다는 의미임. 이후 expenses 에 표시 열을 추가해도 원본 df 에 해당 열이 자동으로 생기지 않음.

> *필터링 후에도 원래 인덱스가 유지됨. 수입 표의 인덱스가 0, 6, 12 인 것은 오류가 아님.*

3.2 전체 수입·지출·남은 금액

```
total_income = income['금액'].sum()
```

```
total_expense = expenses['금액'].sum()
remaining = total_income - total_expense

print(f'전체 수입: {total_income:,}원')
print(f'전체 지출: {total_expense:,}원')
```

```
print(f'남은 금액: {remaining:,}원')
```

지표	금액
전체 수입	6,000,000 원
전체 지출	3,710,000 원
남은 금액	2,290,000 원

sum()은 합계를 계산함. f-string 의 :,는 숫자에 천 단위 쉼표를 붙이는 출력 형식이며, 원래 숫자 데이터를 바꾸지 않음.

해석: 기록된 3 개월 동안 수입이 지출보다 2,290,000 원 많았음. 여기서 '남은 금액'은 기록된 기간의 **수입–지출 차이**이며, 기초 잔액을 반영한 실제 통장 잔액이 아님.

---

4. 새 열 만들기와 조건별 조회

4.1 함수와 apply()로 지출 분류

거래 한 건의 금액이 10 만 원 이상이면 큰 지출, 미만이면 일반 지출로 표시함.

```
def expense_label(amount):
```

```
    if amount >= 100000:
        return '큰 지출'
    else:
        return '일반 지출'

expenses['표시'] = expenses['금액'].apply(expense_label)
```

```
display(expenses[['항목', '금액', '표시']])
```

apply(expense_label)은 금액 열의 각 값에 함수를 적용함. 반환된 결과를 표시라는 새 열에 저장함. 이 예제는 Series.apply()이므로 함수에 표 전체나 행이 아니라 **금액 값 하나씩** 전달됨.

금액 예시	표시
70,000 원	일반 지출
100,000 원	큰 지출
600,000 원	큰 지출

예제 전체에서는 큰 지출이 **11 건**, 일반 지출이 **4 건**임. >=이므로 정확히 100,000 원인 거래도 큰 지출에 포함됨. 이 기준은 학습용 분류 규칙이며 소비의 적절성을 평가하는 기준이 아님.

4.2 금액 조건과 여러 항목 조건

```
big_expenses = expenses[expenses['금액'] >= 100000]
```

```
food_shopping = expenses[expenses['항목'].isin(['식비', '쇼핑'])]

display(big_expenses.head())
```

```
display(food_shopping)
```

isin(['식비', '쇼핑'])은 항목이 식비 **또는** 쇼핑인 행을 선택함. 해당 거래는 총 6 건임.

big_expenses 에는 11 건이 있지만, head()로 출력하면 처음 5 건만 보임. 화면에 보이는 행 수와 전체 행 수를 구분해야 함.

---

5. 항목별 지출 집계와 순위

5.1 같은 항목끼리 묶어서 합산

```
category_total = expenses.groupby('항목')['금액'].sum()
```

```
category_total = category_total.sort_values(ascending=False)

display(category_total)
print('가장 큰 지출 항목:', category_total.index[0])
```

```
print(f'해당 항목 합계: {category_total.iloc[0]:,}원')
```

묶기 → 금액 열 선택 → 합산 → 내림차순 정렬 순서

표현	의미
groupby('항목')	같은 항목끼리 그룹 만들기
['금액'].sum()	각 그룹의 금액 합계 계산
sort_values(ascending=False)	큰 금액부터 정렬
index[0]	정렬된 결과의 첫 번째 항목 이름
iloc[0]	위치 기준으로 첫 번째 합계 값

5.2 집계 결과

순위	항목	3 개월 지출 합계
1	월세	1,800,000 원
2	식비	850,000 원

순위	항목	3 개월 지출 합계
3	쇼핑	550,000 원
4	통신	300,000 원
5	교통	210,000 원

해석: 가장 큰 지출 항목은 월세임. 이 결과는 가장 비싼 거래 한 건을 찾은 것이 아니라, **항목별로 3 개월 금액을 모두 더해 비교한 것**임.

---

6. 월별 요약표 만들기

6.1 날짜 문자열에서 연월 추출

```
df['월'] = df['날짜'].str[:7]
```

```
display(df[['날짜', '월', '구분', '금액']].head())
```

'2026-01-05'의 앞 7 글자는 '2026-01'임. 월 숫자만이 아니라 연도까지 포함해, 서로 다른 해의 같은 달이 섞이지 않도록 함.

> *이 방식은 날짜가 모두 YYYY-MM-DD 형식일 때 사용하는 간단한 방법임. 날짜 형식이 다르면 문자열 자르기만으로 올바른 연월을 얻지 못할 수 있음.*

6.2 pivot_table()로 월별 수입·지출 합산

```
monthly = df.pivot_table(
```

```
    index='월',
    columns='구분',
    values='금액',
```

```
    aggfunc='sum',
    fill_value=0,
)

# 수입/지출 열의 존재와 순서를 명확히 지정
monthly = monthly.reindex(columns=['수입', '지출'], fill_value=0)
monthly.columns.name = None
monthly['남은금액'] = monthly['수입'] - monthly['지출']
```

```
display(monthly)
```

인수·설정	역할
index='월'	월을 행 기준으로 사용
columns='구분'	수입과 지출을 열로 배치
values='금액'	금액을 집계
aggfunc='sum'	여러 거래를 합산
fill_value=0	피벗 결과의 빈 집계 칸을 0 으로 채움
columns.name = None	열 축의 이름인 '구분' 표시를 없앰. 실제 수입·지출 열 이름은 유지

pivot_table()은 거래 상세 데이터를 요약표로 바꾸는 도구임. 기본 집계는 평균이므로, 이 프로젝트에서는 **합계를 구하도록 aggfunc='sum'을 명시**해야 함.

원본의 reindex(...)는 주석 처리되어 있음. 위 정리본에서는 전체 데이터에 수입 또는 지출이 아예 없는 경우에도 해당 열을 만들도록 활성화하였음. 원본 예제의 결과는 변하지 않음.

6.3 월별 결과

월	수입	지출	남은 금액
2026-01	2,000,000 원	1,200,000 원	800,000 원
2026-02	2,000,000 원	1,420,000 원	580,000 원
2026-03	2,000,000 원	1,090,000 원	910,000 원

월별 수입은 같지만 지출은 다름. 2 월의 지출이 가장 많고, 3 월의 지출이 가장 적음. 따라서 3 월에 남은 금액이 가장 큼.

6.4 남은 금액이 가장 큰 달 찾기

```
best_month = monthly['남은금액'].idxmax()
```

```
print('가장 많이 남은 달:', best_month)
```

```
print(f'그 달의 남은 금액: {monthly.loc[best_month, "남은금액"]:,}원')
```

결과는 **2026-03, 910,000 원**임.

- max()는 최댓값을 반환함.
- idxmax()는 최댓값이 있는 인덱스 이름을 반환함. 여기서는 월임.
- loc[월, 열]은 이름을 기준으로 값을 선택

---

7. 그래프와 파일 저장

7.1 항목별 막대그래프

항목별 합계는 크기를 비교하는 데이터이므로 막대그래프로 표현함. 아래 코드는 원본의 한글 폰트 탐색과 영문 대체 처리를 유지한 것임.

```
from matplotlib import font_manager
```

```
plt.style.use('seaborn-v0_8-whitegrid')

font_names = {font.name for font in font_manager.fontManager.ttflist}
font = next((name for name in [
    'Noto Sans CJK KR', 'Noto Sans CJK JP',
    'Malgun Gothic', 'AppleGothic', 'NanumGothic'
] if name in font_names), None)

plt.rcParams['font.family'] = font or 'DejaVu Sans'
plt.rcParams['axes.unicode_minus'] = False

plot_data = category_total.copy()
if font is None:
    plot_data.index = plot_data.index.map({
        '월세': 'Rent', '식비': 'Food', '교통': 'Transport',
        '쇼핑': 'Shopping', '통신': 'Phone'
    })

ax = plot_data.plot(kind='bar', figsize=(8, 4), color='#337D9D', rot=0)
ax.set_title('항목별 지출 합계' if font else 'Expense totals')
ax.set_xlabel('항목' if font else 'Category')
ax.set_ylabel('금액 (원)' if font else 'Amount (KRW)')

plt.tight_layout()
plt.savefig(OUTPUT / '항목별지출.png', dpi=150)
plt.show()
```

```
plt.close()
```

kind='bar'는 막대그래프, rot=0 은 항목 글자를 회전하지 않는 설정임. tight_layout()은 여백을 조정하고, savefig()는 이미지 파일을 저장함. dpi 는 저장 이미지의 해상도 설정

한글 폰트가 없으면 그래프만 영문으로 표시하고, 분석 결과의 한글 항목명은 유지함. 스타일 적용 후 폰트를 설정해야 스타일이 폰트 설정을 덮어쓰는 일을 피할 수 있음.

y 축에 1e6 처럼 표시되면 눈금 값에 1,000,000 을 곱해 읽음. 전체 숫자로 표시하려면 원본의 주석 처리된 코드를 사용할 수 있음.

```
ax.ticklabel_format(axis='y', style='plain')
```

이 설정은 plt.tight_layout() 전에 넣음.

7.2 CSV 저장

```
category_total.rename('지출합계').to_csv(
    OUTPUT / '항목별지출.csv', encoding='utf-8-sig'
)
monthly.to_csv(OUTPUT / '월별요약.csv', encoding='utf-8-sig')
print('저장 완료:', OUTPUT.resolve())
```

rename('지출합계')는 집계된 Series 의 이름, 즉 저장할 값 열의 이름을 지정함. utf-8-sig 는 엑셀에서 한글을 읽을 때 호환성을 높이기 위한 인코딩

CSV 는 기본적으로 인덱스도 저장함. 이 예제에서는 항목별 표의 '항목'과 월별 표의 '월'이 인덱스이므로, 인덱스를 함께 저장하는 것이 의미가 있음.

저장 파일	내용
outputs/항목별지출.png	항목별 지출 막대그래프
outputs/항목별지출.csv	항목별 지출 합계
outputs/월별요약.csv	월별 수입·지출·남은 금액

resolve()는 실제 저장 위치의 절대 경로를 보여 줌. 원본에 남아 있는 Windows 경로는 원본을 실행한 컴퓨터의 경로이며, 수강생의 저장 위치는 실행 환경에 따라 달라짐.

---

8. 미니 실습: 기본에서 확장으로

먼저 문제만 제시하고, 학습자가 작성한 뒤 아래 정답 예시와 비교함. 실습 전까지 앞의 코드가 실행되어 df, expenses, monthly 가 만들어져 있어야 함.

8.1 실습 문제와 예상 결과

번호	문제	사용하는 개념	예상 결과
1	쇼핑 내역만 선택해 shopping 에 저장	조건 필터링	3 건
2	쇼핑 금액 합계를 shopping_total 에 저장	sum()	550,000 원
3	20 만 원 이상은 '확인', 미만은 '통과'로 분류해 review 에 저장	apply(), 조건식	확인 7 건, 통과 8 건
4	70 만 원 이상 남은 달을 saved_months 에 저장	조건 필터링	1 월, 3 월
5	월별 남은 비율을 remaining_percent 에 저장	열 간 계산	40.0%, 29.0%, 45.5%
6	1 월과 2 월 거래를 two_months 로 합치기	concat()	12 행
7	매월 70 만 원 목표와 실적 비교	merge()	1 월·3 월 달성, 2 월 미달

8.2 미니 실습 정답 예시: 1~5

```
# 실습 1: 쇼핑 내역 선택
```

```
shopping = expenses[expenses['항목'] == '쇼핑'].copy()
```

```
display(shopping)

# 실습 2: 쇼핑 금액 합계
shopping_total = shopping['금액'].sum()
print(f'쇼핑 합계: {shopping_total:,}원')

# 실습 3: 20만 원 이상 표시
review = expenses['금액'].apply(
    lambda amount: '확인' if amount >= 200000 else '통과'
)
display(review)

# 실습 4: 70만 원 이상 남은 달
saved_months = monthly[monthly['남은금액'] >= 700000].copy()
display(saved_months)

# 실습 5: 수입 대비 남은 금액의 비율
remaining_percent = monthly['남은금액'] / monthly['수입'] * 100
```

```
display(remaining_percent.round(1))
```

lambda 는 간단한 함수를 한 줄로 쓰는 표현임. 앞서 만든 expense_label()처럼 일반 함수로 작성해도 됨. 실습 2 는 실습 1 의 결과를 재사용하였으며, 원본처럼 expenses 에서 쇼핑을 다시 필터링해 합산해도 같은 결과

남은 비율의 계산식은 **남은 금액 ÷ 수입 × 100** 임. 실제 자료에서 수입이 0 이면 별도 처리가 필요함. 계산 중 생기는 소수 표현 차이는 round(1)로 보기 좋게 표시할 수 있음.

8.3 확장 실습 6: concat()으로 행 이어 붙이기

```
january = df[df['월'] == '2026-01'].copy()
```

```
february = df[df['월'] == '2026-02'].copy()

two_months = pd.concat([january, february], ignore_index=True)
```

```
display(two_months)
```

concat()은 기본적으로 표를 **세로 방향으로 이어 붙임**. 1 월 6 건과 2 월 6 건을 합치므로 결과는 12 행임. ignore_index=True 는 결과의 행 인덱스를 0 부터 다시 매김.

원본 실습 제목은 '월별 파일을 하나로 붙이기'이지만, 실제 코드는 파일을 읽지 않고 이미 만들어진 df 에서 두 달의 데이터를 골라 합침.

8.4 미니 실습 7: merge()로 목표 연결

```
goal = pd.DataFrame({
    '월': ['2026-01', '2026-02', '2026-03'],
    '목표금액': [700000, 700000, 700000],
})

comparison = monthly.reset_index().merge(
    goal, on='월', how='left', validate='one_to_one'
)
comparison['차이'] = comparison['남은금액'] - comparison['목표금액']
```

```
display(comparison[['월', '남은금액', '목표금액', '차이']])
```

월	남은 금액	목표금액	차이	해석
2026-01	800,000 원	700,000 원	+100,000 원	목표보다 10 만 원 더 남음
2026-02	580,000 원	700,000 원	−120,000 원	목표보다 12 만 원 부족
2026-03	910,000 원	700,000 원	+210,000 원	목표보다 21 만 원 더 남음

reset_index()는 월 인덱스를 일반 열로 바꿈. merge()는 공통 열인 월을 기준으로 목표금액을 연결함. how='left'는 왼쪽 실적 표의 월을 모두 유지하며, validate='one_to_one'은 양쪽 표에서 월 값이 중복되지 않는지 검사함.

병합 검증 옵션	조건
one_to_one	양쪽 키가 모두 고유
one_to_many	왼쪽 키는 고유, 오른쪽 키는 중복 가능

병합 검증 옵션	조건
many_to_one	왼쪽 키는 중복 가능, 오른쪽 키는 고유
many_to_many	양쪽 키 중복 허용. 키 고유성을 검증하지 않음

목표표에 어떤 월이 빠져 있으면 병합 결과의 목표금액은 결측값이 됨. one_to_one 은 중복을 검사하는 옵션이지, 모든 월의 목표가 있는지 확인하는 옵션이 아님.

핵심 비교: 이 예제에서 concat()은 같은 구조의 거래 표에 **행을 추가**하고, merge()는 월이라는 공통 기준으로 **다른 표의 정보를 연결**

---

9. 주의사항

9.1 '남은 금액'과 실제 잔액 구분하기

남은 금액은 분석 기간의 수입–지출임. 실제 잔액을 구하려면 기초 잔액과 모든 입출금이 필요함. 또한 월별 남은 금액은 전월 잔액을 더한 누적 잔액이 아님.

9.2 실제 거래 내역의 정리 규칙 필요성

실제 통장·카드 자료에는 환불, 계좌 간 이체, 카드 결제와 카드대금 출금의 중복 등이 있을 수 있음. 예제처럼 단순 합산하기 전에 무엇을 수입과 지출로 볼지 정해야 함. 결측값이 없다는 것만으로 데이터가 정확하다고 판단해서는 안 됨. 구분의 오탈자, 금액의 숫자 자료형·부호, 중복 거래도 확인해야 함.

9.3 날짜 문자열 자르기의 전제 이해하기

원본의 str[:7]은 고정된 날짜 형식을 전제로 함. 날짜 형식이 다르거나 날짜 계산이 필요할 때는 다음처럼 날짜 자료형으로 변환하는 방법을 소개할 수 있음. 이는 원본의 필수 단계가 아니라 보완 설명

```
dates = pd.to_datetime(df['날짜'], format='%Y-%m-%d')
```

```
df['월'] = dates.dt.strftime('%Y-%m')
```

9.4 피벗의 빈 칸과 없는 행·열 구분하기

fill_value=0 은 피벗 결과의 빈 집계 칸을 채우지만, 전체 데이터에 한 번도 없는 수입·지출 열이나 거래가 전혀 없는 월을 자동으로 만들지는 않음. 열 보완에는 앞서 제시한 reindex()를 사용할 수 있음. 없는 월을 0 으로 채울 때도 '거래가 실제로 없었음'과 '자료가 누락됨'을 구분해야 함.

9.5 실행 순서와 화면 출력 구분하기

노트북은 앞에서 만든 변수를 뒤의 셀에서 사용함. 원본에는 실행 번호가 건너뛰거나 실행 기록이 없는 셀이 있으므로, 수업 전에는 커널을 재시작하고 위에서 아래로 모두 실행해 확인함. expenses 에 추가한 표시 열은 df 에 자동 반영되지 않고, 그 이후 df 에 추가한 월 열도 기존 expenses 에 자동 반영되지 않음.

9.6 빈 데이터와 동률 주의하기

category_total.index[0]은 지출 데이터가 존재한다는 전제가 있음. 지출이 없으면 먼저 빈 결과를 확인해야 함. idxmax()는 최댓값이 여러 개일 때 첫 번째 인덱스를 반환하므로, 모든 공동 1 위를 찾는 기능과는 다름.

---

10. 핵심 요약

분석 작업	핵심 코드·개념
표 만들기	pd.DataFrame()

분석 작업	핵심 코드·개념
기본 점검	head(), len(), value_counts(), isna()
조건 선택	df[조건], isin()
합계 계산	sum()
함수로 새 열 만들기	Series.apply()
항목별 집계·순위	groupby(), sort_values()
월별 요약	str[:7], pivot_table()
최댓값의 위치 찾기	idxmax(), loc[]
그래프·파일 저장	plot(), savefig(), to_csv()
표 이어 붙이기	concat()
기준 열로 정보 연결	merge()

학습 확인 항목

① 양수 금액 기록 시 수입과 지출의 구분 방법

② 필터링 후 행 번호가 연속적이지 않은 이유

③ 이 프로젝트에서 groupby()와 pivot_table()이 만드는 결과

④ max()와 idxmax()의 반환값 차이

⑤ concat()과 merge()의 사용 상황 차이

⑥ 남은 금액의 양수 여부만으로 실제 통장 잔액을 알 수 없는 이유

답변 요약: ① 구분 열로 나눔 ② 원래 인덱스가 유지됨 ③ 항목별 지출 합계 / 월별 수입·지출 표 ④ 최댓값 / 그 값의 인덱스 ⑤ 거래 행 이어 붙이기 / 월을 기준으로 목표 연결 ⑥ 기초 잔액과 누락·중복 거래 등이 없어야 실제 잔액 계산 가능.

마무리

> *거래 내역을 표로 만들고, 조건에 맞는 데이터를 고른 다음 항목별·월별로 합산*
> *핵심은 함수를 외우는 것보다 '어떤 분석 목적에 답하기 위해, 어떤 기준으로 데이터를 선택하고 묶을 것인가'를 이해하는 것. 같은 흐름을 매출, 성적, 재고 같은 다른 표 데이터에도 적용할 수 있음.*